AI-assisted coding, assessment design, and the agentic AI era

If students can use AI to write code, what are we really assessing?

James Kermode · School of Engineering, University of Warwick

6 May 2026 · Oxford Brookes · AI & Data Analysis Network

Personal experience as a Warwick academic and HetSys CDT director — not the institutional view of the University of Warwick. Warwick's AI strategy is being developed separately.

Talk Thesis

If students can use AI to write code, what are we really assessing?

*One useful way forward: treat **coding as formative**, and **understanding as summative**.*

- This talk argues that split, and gives a worked example — **mograder** — of what it can look like in practice.
- Plus the patterns and anti-patterns of using Agentic Coding tools well for your own work.
- Audience is mixed: discipline-specific examples where useful, but the assessment question cuts across.

A spectrum of AI coding tools

Vibe coding — *increasing context · increasing autonomy · increasing oversight responsibility* — **Agentic engineering**

ChatGPT in browser

- Copy/paste both ways
- No project context
- You run the code
- **ChatGPT, Claude.ai**

Autocomplete in editor

- Inline tab-completion
- Current-file context
- You drive every keystroke
- **Copilot, Cursor tab**

Full AI editor

- Chat edits files in repo
- Codebase-aware context
- You approve diffs in IDE
- **Cursor, Windsurf**

Agent

- Runs commands, iterates
- Full project context (`CLAUDE.md`)
- Review at task boundaries
- **Claude Code, Codex, Gemini CLI**

*Same model can sit behind any of these — the difference is the **harness**: how much context it sees, whether it can act, and where the human reviews. Willison: "Agentic Engineering represents the other end of the scale — professional engineers using coding agents to amplify their existing expertise." Case studies that follow use Claude Code; the patterns themselves are agnostic to which agentic harness you prefer.*

How an LLM chatbot works

Pseudocode Sketch

```
messages = [system_prompt, task]

while not done:
    reply = model.generate(messages)

    messages.append(reply.text)
    display(reply.text)
    done = await_user()
```

Key properties

- **System prompt** – persistent instructions set by the application (role, tone, safety, format)
- **`messages`** – full conversation history, replayed on every call; the model has no memory between sessions
- **Context window** – finite tokens per turn (200k-1M tokens, ~150k-750k words)
- **Sampling** – `generate()` picks tokens probabilistically; same input can give different replies (temperature, top-p)
- **Knowledge cutoff** – no access to events after training ends
- **No actions** – model cannot run code, read files, or browse the web – only text in, text out

ChatGPT, Claude, Gemini etc. all basially work like this.

Agentic Loop

Pseudocode Sketch

```
messages = [system_prompt, CLAUDE_MD, task]
tools    = [Bash, Read, Edit, Write, Grep,
            Glob, WebFetch, WebSearch, Task]
while not done:
    reply = model.generate(messages,
                           tools=tools)
    if reply.tool_calls:
        for call in reply.tool_calls:
            out = run_tool(call.name,
                           call.args)
            messages.append(
                tool_result(call, out))
    else:
        messages.append(reply.text)
        display(reply.text)
        done = await_user()
```

Examples of Tools

- **Bash** – run a shell command (`pytest -xvs`, `uv run …`, `git log`, `make`)
- **Read** / **Write** / **Edit** – create or open a file, or do a find-and-replace
- **Grep** – regex-search (e.g. `def test_energy` across `**/*.py`)
- **Glob** – enumerate files by pattern (`src/**/*.py`)
- **WebFetch** – pull a specific URL (docs, GitHub issues, papers)
- **WebSearch** – look up recent/obscure info (post-training data, niche APIs, error message searches)
- **Task** – spawn a sub-agent with its own context (“audit this PR”, “find all callers of X”). Can use a cheaper model for sub-tasks.

The agent is only as capable as its tools. Add an MCP server and it gains a new action; remove `Bash` and it can no longer run code. You can teach it to use new CLI tools (or even write new ones which are agent-friendly, with explanatory docs).

Agentic vs. Vibe Coding – Key Differences

- Agent = model + harness that **executes** code and feeds results back
- Core loop is 200 lines of code, not magic (MIT Missing Semester, 2026)

- **Willison: “Writing code is cheap now”.**
- Typing code into a computer is now almost free.
- Delivering **good** code, which is tested, correct, maintainable, is not.

In most disciplines, “correct” means more than “the tests pass”:

- Code must also be correct **in your domain’s sense** – physically, causally, pedagogically.
- No auto-generated test suite fully captures that: that is **your** job.

Common Objections

“AI writes code that doesn’t compile” Zero-shot, no context, wrong language version: yes, often. With a `CLAUDE.md` specifying your Python/Julia version, package environment, and a working example to follow: almost never. The agent executes and iterates – it sees the compiler error.

“Fortran coverage is low” Correct. Training data is dominated by Fortran 77; modern Fortran 2018 features and fixed/free format confusion are failure modes. My QUIP work in this talk was CI and Python glue, not extensively touching the Fortran. Know your tool’s limits.

“Zero-shot failures on niche APIs” Correct. Zero-shot with no context for a specific LAMMPS fix style or ASE calculator is unreliable. Solution: provide a working example from your codebase. The agent recombines; it doesn’t (shouldn’t) invent.

“I tried it and got rubbish” Almost certainly towards vibe coding end of spectrum: without `CLAUDE.md`, working examples, or iterative feedback. The same person wouldn’t hand a junior developer a task with no briefing and be surprised at the output. Capable models also help — recent flagship models from Anthropic, OpenAI, and Google all clear the bar; older or distilled checkpoints often don’t.

Harder Objections

“It made me slower, not faster” A rigorous RCT (METR, July 2025) found experienced open-source developers were **19% slower** with AI tools. The finding is real for tasks where the developer already knows exactly what to do. Gains appear on unfamiliar territory, bridging expertise gaps, and maintenance work. Track time honestly.

“It’s just regurgitating training data” Partly fair. For novel algorithms, you are the source of novelty: the agent implements what you specify. That is the spec-first pattern: you design, it codes. The Anthropic C compiler rewrite attracted “code laundering” criticism; for original research, the question is whether your idea/spec is novel.

Skill atrophy: the paradox of supervision Anthropic internal study (Dec 2025): developers scored **17% lower** on code comprehension tests when using AI to learn a new library. Paradox: effective use of Agentic AI requires supervising Claude, which requires the very skills that atrophy. Practise without AI too!

Data privacy and reproducibility Your code goes to Anthropic’s servers – OK for open source code, otherwise not. For sensitive data, local models (Ollama + open weights) are improving rapidly. Hybrid approaches possible.

Pedagogical Tensions

Struggle as pedagogy

- Hitting a confusing error and working through it **is** the learning.
- Friction builds intuition.
- Researchers who never debug never develop a mental model of failure modes.

Reducing friction

- Remove obstacles that aren't educational.
- Better tools let people spend cognitive load on the science, not boilerplate.
- We don't code in assembly any more!
- AI assistance is now a professional reality in many domains

The question is not whether to use AI tools – it is which friction is worth keeping.

Which friction is worth keeping?

Worth keeping

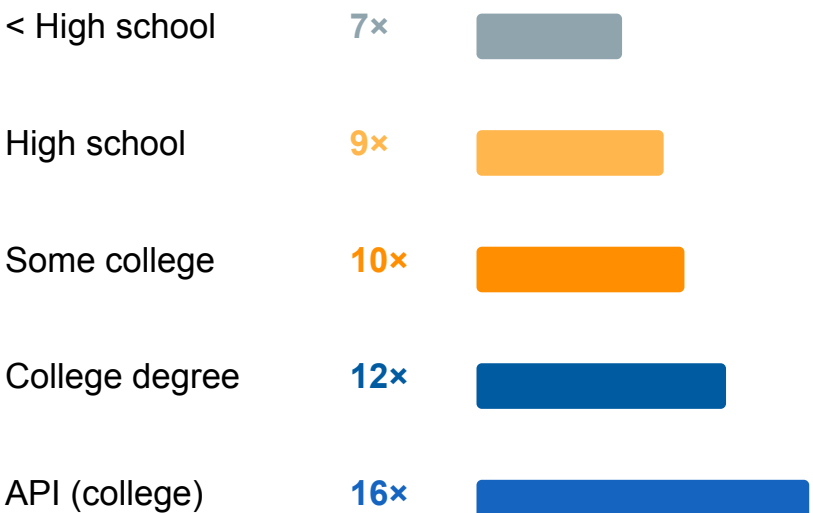
- Debugging logic errors in your own code
- Understanding **why** a numerical method converges or doesn’t
- Choosing the right algorithm for the problem
- Interpreting results physically
- Reviewing/owning every line in a paper or PR

Worth removing

- Boilerplate file I/O and plotting scaffolding
- Looking up library API signatures
- Translating a known algorithm into working syntax, or from one language to another
- Writing tests for behaviour you already understand
- Formatting and docstrings

Speedup grows with expertise – Anthropic Economic Index, Jan 2026

Measured speedup on Claude.ai by task complexity *(years of schooling required to understand the task)*



*Source: Anthropic Economic Index, Jan 2026 (~2M Claude conversations, Nov 2025) – **vendor-published, treat as directional**. Speedup = human-alone time / human-with-AI time.*

What this means for us as researchers

- More expertise you bring, more you gain.
- A PhD student at the frontier gets more leverage than someone doing routine tasks.

- Contradicts “AI levels the playing field” narrative
- Domain knowledge is amplified, not devalued
- The METR 19% slowdown: low end of this curve
- **Caveat**: data tops out at college-level tasks – no direct measurements of PhD-level work. Trend suggests further amplification; nobody has proved it.

Success rate falls with complexity (70% → 66%). Human oversight becomes more important, not less.

mograder — a worked example

Formative coding, summative understanding

mograder

mograder is an autograder for coding assignments. It provides fast formative feedback on code correctness, and allows summative assessment of understanding via written/oral components.

Motivation I want to teach postgraduate students to use AI tools effectively, not ban them.

The task

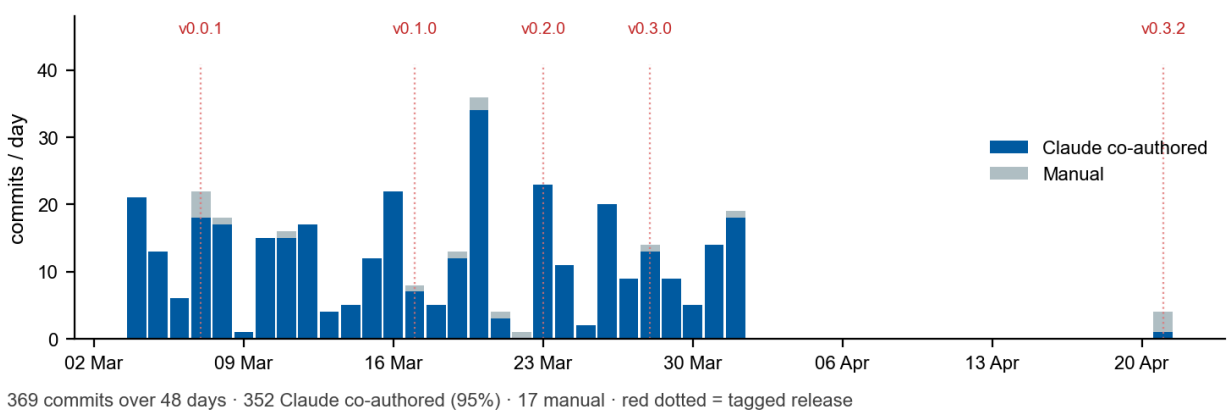
- Generate / autograde / validate assignments
- Student and grading web interfaces
- Moodle API integration
- Robust security model
- Comprehensive docs · PyPI release · CI/CD

The setup

- Claude Code bare metal on Mac and Linux
- Close supervision – reviewing every change
- Git as the safety net and progress log
- Iterative: short tasks, verify, commit, repeat

Development process

- I wrote almost no code by hand. Almost every line was generated by Claude Code, and reviewed by me.
- **369 commits over 48 days** · 95% Claude co-authored, 5% manual touch-ups · 30 active days (≈12 commits/day when working on it); busiest day: **36 commits**
- **4 days from empty repo to first tagged release**; released on PyPI after 13 days; 36 tagged versions
- **19k lines of Python** (53 modules) + **16k lines of tests** (482 tests, ~25 tests / kLOC src) + 2.5k lines docs



What the agent was good at

- **Boilerplate-heavy infrastructure** – FastAPI routes, SQLite schema, Moodle REST calls, CI/CD YAML configuration, Python packaging.
- **Verifiable tasks** – “does `pytest` pass?”, “does `pip install` work?”, “does the Moodle token round-trip?”, “does the example notebook run/autograde without error?”
- **Recombining known patterns** – PEP 723 metadata + cell-hash embedding merged two things it already understood
- **Documentation and tests** – given a spec, write docstrings and unit tests first

The agent is a very fast, tireless junior developer with broad knowledge and zero physical intuition.

What required human judgement

- **Security model design** – six-layer defence-in-depth was a deliberate choice (resource limits, timeouts, static safety checks, temp directory isolation, bubblewrap sandbox, integrity checking). The agent would have stopped at “run in a temp dir”.
- **Assessment philosophy** – formative coding, summative written and oral components, students must explain code with their name on it. Practical experience with similar tools.
- **Architecture** – Marimo over Jupyter, sidecar JSONL over HTML parsing to extract machine-readable metadata, write-ahead logging (WAL) mode for SQLite operations.
- **Catching agent errors** – plausible-looking but broken async logic; incorrect Moodle API endpoints needed iterative testing and careful review to catch.

Agents can deal with the ‘How’ but not the ‘What’ or the ‘Why’. You cannot delegate the decisions that define what the software is for!

mograder — formative / summative separation

Formative — the code

- **Release notebook**: students receive the assignment with solutions stripped and check cells pre-populated
- **Instant feedback**: checks run as each cell is completed; an incorrect answer gets partial credit + targeted feedback, not pass/fail
- **Tamper-proof**: edit a check cell and the hash mismatch is detected
- **Workshop mode**: hosted in a static HTML page via WASM: works in a browser, no server or VLE. Instructors can release model solutions live.
- **Low-stakes, fast turnaround**: automated tests.

Summative — the understanding

- **Written defence**: students explain their approach — why this algorithm, where it might fail
- **Oral mini-viva**: 10-15 minutes, student walks the marker through their submitted code. Random sample of students, allocated after submission.
- **The student's name is on it.** It doesn't matter how the code was written; they still have to defend it.
- **Higher-stakes, hard to delegate**: human judgement.

*mograder allows delegating **pattern-matching** — did the code produce the right answer? — to the machine, but keeps the **human question** — did the student understand? — for humans.*

Mograder Demo

- A **release notebook** – what students receive (solutions stripped, check cells present)
- **Formative checks** run as cells are completed
- An **incorrect** answer gets partial credit + targeted feedback rather than pass/fail
- **Tamper-proof submission**: edit a check cell and the hash mismatch is caught
- **Instructor dashboard**: all assignments, submissions and marks in one view
- **Workshop demo**: example workshop-mode, fully formative and hosted in a static HTML page (via WASM). Instructors can release model solutions live during workshops.

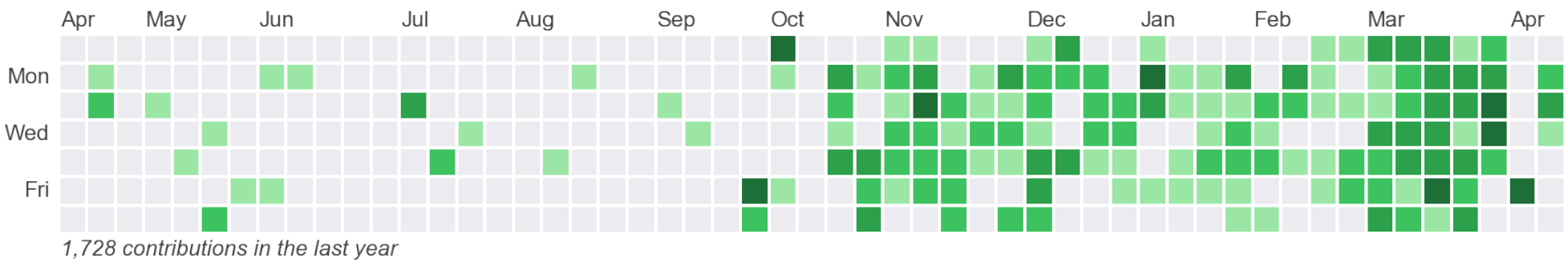
Student dashboard: `mograder-demo.jrkermode.uk`
Instructor dashboard: `mograder-demo.jrkermode.uk/grader`
Workshop demo: `jameskermode.github.io/mograder/dashboard/notebooks/demo-workshop.html`

Further examples

A quick tour of my recent AI-assisted codebases

Can you spot when I started using Claude?

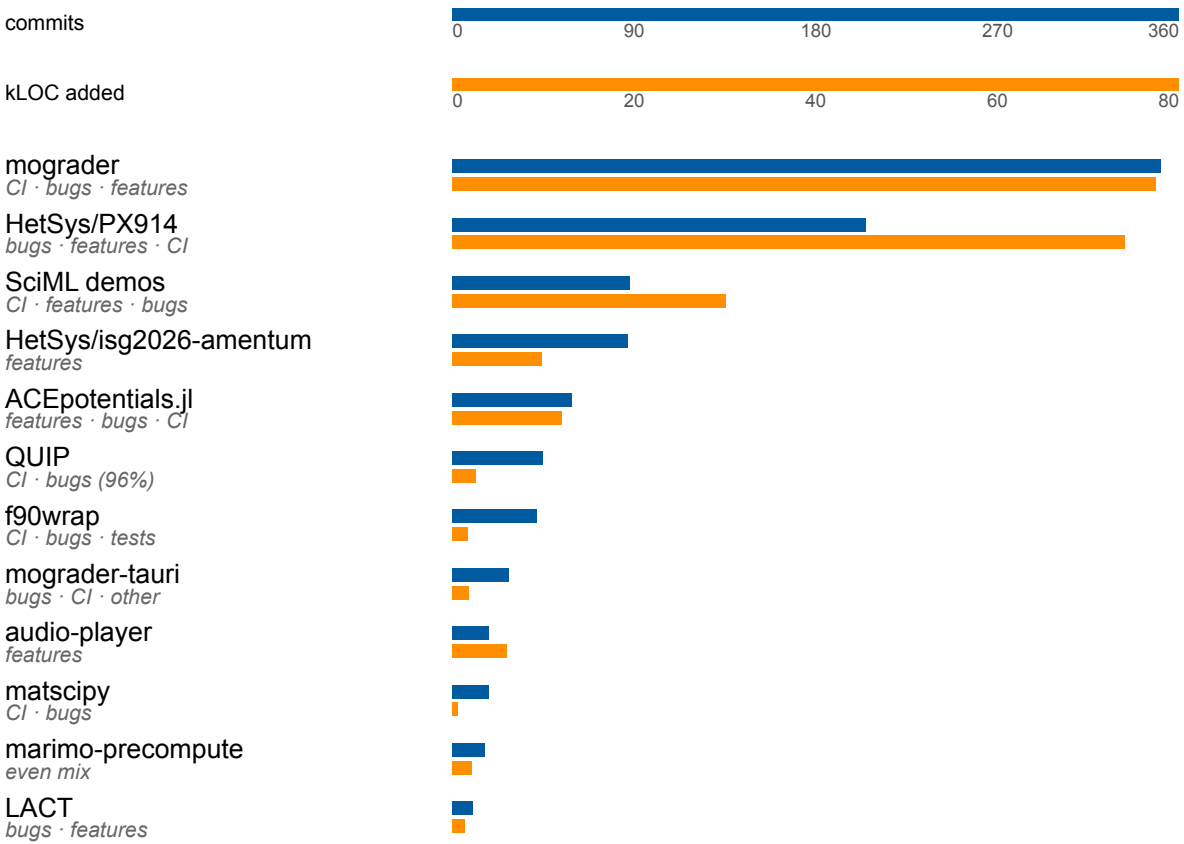
GitHub contribution heatmap (jameskermode · Apr 2025 – Apr 2026)



1,728 contributions in the last year

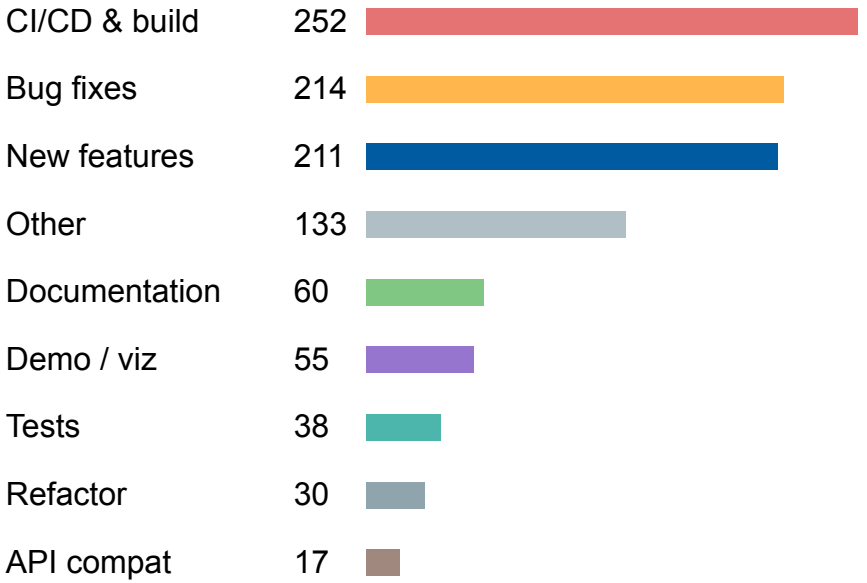
1k Claude co-authored commits · 246k lines of code

By repository — commits (blue) and kLOC added (orange)



26 repos · `git log --numstat` · simulation data excluded · Mar 2025–Apr 2026

By contribution type



25% is CI/CD + build infrastructure – the unglamorous maintenance work that would otherwise never get done.

***Cost log**: ~£400 over 6 months on Claude Max 5× (~$2,480 at metered rates). Specialist knowledge made this hard to delegate – counterfactual is "not done", not "done by someone else".*

Second-order effects: Atomistica

- **Atomistica:** (`github.com/atomistica/atomistica`) Fortran library of interatomic potentials (ASE + LAMMPS compatible) maintained by Lars Pastewka in Freiburg.
- `numpy.distutils` removed in Python 3.12 – Atomistica's build system was broken
- I needed it for teaching, and thought it would be a good test of what Claude Code could handle on an unfamiliar Fortran codebase

Oct 2025: my meson PR

Migrated from `numpy.distutils` to Meson + `meson-python`. PR description: **"I did it using Claude Code so some careful testing is required before merging."** Merged with final fixes, Oct 20.

Dec 2025: Lars starts C++ rewrite

PR 54: **"Complete rewrite of Atomistica in modern C++."** 72 files · 21,623 lines of new C++ · full reimplementation of all potentials, integrators, neighbour lists.

Six weeks between "fix our build system" and "rewrite 15 years of Fortran in modern C++" – the willingness to attempt ambitious transformations spreads, across disciplines as well as codebases.

The wider landscape: AI agents for science

Anthropic: C Compiler (Feb 2026)

16 parallel agents, ~2,000 sessions, $20k → 100k lines of Rust compiling Linux 6.9. **But** Carlini: **"The thought of programmers deploying software they've never personally verified is a real concern."**

Google: AI Co-Scientist (Feb 2025)

Multi-agent Gemini 2.0 generates novel hypotheses. Drug-repurposing candidates for AML confirmed in vitro; independently reached an AMR hypothesis that took Imperial years. Beyond literature review: novel suggestions.

NVIDIA ALCHEMI (Dec 2025)

NVIDIA Inference Microservices (NIM) for batched conformer search + MD using MACE-MPA-0, TensorNet, AIMNet2. Universal Display: billions of OLED candidates, 10,000x faster than DFT on CPU. Open-source agentic coding toolkit.

Argonne/RSC: agent (2026)

Digital Discovery 2026: autonomous MD pipelines – Atomsk, MLIP web-mining, LAMMPS on HPC, OVITO/Phonopy post-processing. All from natural language. "Run the right simulation" remains human territory.

Agents handle mechanical complexity; domain expertise – what to simulate, whether results make sense – remains the human contribution.

Patterns that work

Four tool-agnostic practices with evidence behind them

Pattern 1 – Hoard things you know how to do

Simon Willison:
“Knowing something is theoretically possible is not the same as having seen it done yourself.”

- Build a personal library of **working code examples** in your domain
- For me this means things like: canonical Python examples, code input templates, JAX jit patterns, file readers, neural network inference scripts
- Reference them in `CLAUDE.md` or in prompts: “use `~/examples/ace_calc.py` as a pattern for this interface”

Agents only need you to figure out a trick once. After that they can recombine it into any similarly shaped future problem, including ones you haven’t thought of yet.

Pattern 2 – Spec first, implement second

- Write the algorithm in pseudocode or maths before opening a session
- Validate the spec yourself: conserved quantities? edge cases? units?
- Start a new session to implement – clean context, written spec as the prompt, in planning mode first to force design negotiation before any code is written

```
# SPEC.md — Score moderation
Input:  raw 1D marks;  target = (mean, std) OR empirical CDF
Output: moderated marks (same length, same shape)
Choose method: linear (mean+std match), quantile (CDF match), or piecewise linear (fix anchor marks).
Test invariants:
  - Spearman(raw, moderated) == 1.0        # rank preserved
  - monotone non-decreasing over full mark range
  - |mean(moderated) - target.mean| < 0.1
  - endpoints fixed if configured: 0 -> 0, 100 -> 100
```

Never ask an agent to design the algorithm **and** implement it in one pass.

Pattern 3 – Red/green TDD

- Write tests that **fail for the right reasons** before any implementation
- The agent's job is then unambiguous: make the tests pass

For scientific computing, this **forces** you to encode physical correctness explicitly:

```
def test_energy_conservation():
    traj = run_md(n_steps=1000, ensemble='NVE')
    drift = max(traj.energies) - min(traj.energies)
    assert drift < 1e-4  # eV, LJ argon @ 94K
```

- **You** must specify the tolerance – the agent cannot guess it
- The test becomes the documentation of what "correct" means

Willison: "TDD produces more succinct and reliable agent output with minimal extra prompting"

Pattern 4 – Manage context obsessively

Context degradation is the primary failure mode. Broad consensus is that output quality starts dropping at ~40% of the context window.

- `CLAUDE.md`: short, universally applicable – loads on **every** session
- `/clear` between tasks; `/compact` at natural checkpoints (commits, test passes)
- Similar tools available for other Agentic Coding systems
- Maintain `PROGRESS.md` – update at session end, paste in at session start (or use `git log`)

```
# PROGRESS.md
Last session: Zygote AD path implemented, all tests pass
Next: benchmark vs finite-differences on W (110) surface
Known issues: periodic boundary edge case in test_forces_pbc
```

Demo – Claude Code live

Example but somewhat realistic teaching tool

Claude Code demo – score moderation from scratch

The prompt:

```
Plan then implement a score-moderation utility in Python. Input: raw marks from a cohort (1D array) and a target distribution specified either as (mean, std) or as a target empirical CDF. Output: moderated marks that match the target. Write tests that verify: (i) rank order is preserved exactly (Spearman correlation between raw and moderated = 1.0); (ii) moderated mean is within 0.1 of target mean; (iii) endpoints are fixed if configured (0 → 0, 100 → 100); (iv) the transformation is monotone non-decreasing across the full mark range. Once validated, wrap it to read a Moodle gradebook CSV, produce moderated marks, and emit diagnostic plots (Q-Q plot of raw vs moderated, histogram overlay). Use uv + pyproject.toml, red/green TDD with pytest, numpy + scipy + matplotlib. Ask any clarifications first.
```

Why the prompt is shaped this way:

- **Pattern 2 — spec first.** Force the agent to surface the method choice and its downstream decisions (endpoint anchoring, CDF input shape, CSV column discovery) before writing code.
- **Pattern 3 — red/green TDD.** Four layered invariants — rank, mean, endpoints, monotonicity. **You** decide each tolerance; the agent shouldn't guess them.
- **Staged build-up.** Get the moderation kernel correct in isolation, **then** wire it to the gradebook CSV and plots.
- **Pattern 1 — hoard.** Name the known-working approaches: Spearman, Q-Q, numpy/scipy/matplotlib.
- **Pattern 4 — tooling & context.** Conventions baked into the spec (and `CLAUDE.md`) so the agent doesn't reinvent them.

Anti-patterns

Where it's especially easy to go wrong

Anti-pattern 1 – Unreviewed code on collaborators

Willison: "If you open a PR with code you haven't validated yourself, you're delegating your actual work to other people who could have prompted an agent themselves."

In a research group context:

- Don't push simulation code to a shared repo without running it and understanding it
- Review the **PR description** too: agents write convincing summaries
- Include evidence of testing: a benchmark plot, diff against reference data, timing numbers

The principle: You are responsible for code with your name on it, regardless of how it was produced: in a paper, a PR, or your thesis.

Anti-pattern 2 – Trusting output you can't verify

The software version: agent writes plausible-looking but broken async logic.

Discipline-specific versions are more dangerous:

- **Data analysis**: a regression that runs without error but inverts signs on a categorical coding – numbers are produced, conclusions invert
- **Science / engineering**: wrong units in a physical calculation
- **Any domain**: library API that “works” but with semantics subtly different from expected

> *Mineault (2026): “A clear skill from scientific training is the ability to call bullshit. Have the agent generate lots of diagnostic plots – individually low-utility, collectively they convince you the data is correct.”*

The agent has no notion of disciplinary plausibility. That’s up to us.

Anti-pattern 3 – Stateful notebooks

Mineault (2026): "Jupyter notebooks don't play well with agents: plots are embedded as base64, eating context, and agents don't know the state of your kernel."

Jupyter problems

- Hidden kernel state
- Base64 plots bloat context window
- Cell execution order ambiguity
- Agent can't verify its own output

Solutions

- **Marimo**: reactive execution, no hidden state
- `/marimo-pair` agent tool to edit and run cells
- CLI scripts + PNGs for data pipelines
- Notebooks only for final, linear narrative

Anti-pattern 4 – Over-engineering by default

Models are trained to be agreeable. The agent will never tell you a feature is unnecessary, never say "you're done", never suggest you stop. Combined with build cost being near-zero, this creates a strong pull toward over-engineering.

Symptoms

- Speculative features no one asked for
- Defensive code / fallbacks for cases that can't happen
- Premature abstractions and helper layers
- "While we're here…" scope creep
- Endless polish – the agent always finds one more thing

Antidotes

- Spec the **stop condition** up front; declare done when the spec is met
- Treat agreeable suggestions skeptically – push back, ask "do we need this?"
- Periodically prune: delete unused code, undo speculative abstractions
- Resist the temptation of "just one more iteration"

Outlook – implications for research and teaching

- **The substitution ratio.** ~6 months of infrastructure work for ~£400 of Claude Max subscription vs ~£30–60k of postdoc-equivalent labour. Close to two orders of magnitude.
- **“Substitution” isn’t quite the right frame.** Most of this work needed specialist domain knowledge that couldn’t be cleanly delegated. Most would simply not have been done – closer to “new capability” than “redirected costs”.
- **New grant line item:** “AI tooling” alongside compute costs, with consequences for how RSE/PDRA costs are justified and how their day-to-day roles develop.
- **Where do early-careers learn?** Unglamorous CI / build / maintenance and small new features was always how junior researchers built expertise. If agents do it, the training pipeline needs reworking (who will supervise the agents?)
- **Domain expertise is the bottleneck.** Leverage shifts to people who know the field, and what is possible.
- **None of these have settled answers.** We are running an uncontrolled experiment in real time!

Back to Struggle-as-Pedagogy

“Using the tools proficiently is feasible when you have gone through the hard work of writing your own code by yourself, failing repeatedly, picking yourself back up; I don’t know of another way of getting to that level of metacognition.

How do you develop that when the AI is making the mistakes for you, invisibly?

This is likely to be a challenge for junior researchers as the tools get commoditized.“

— *Patrick Mineault,* ***Claude Code for Scientists*** *(Jan 2026)*

Open question:

How do we help new researchers develop metacognition about code when failure is abstracted away?

No settled answer - here’s my partial idea…

Provisional Advice for Lecturers

For your own prep and marking

- Delegate: conversion between handouts and slide decks, worked-example generation, first-draft marking rubrics, synthetic test data, feedback templates
- Don’t delegate: grade judgements on individual students, exam writing, references, confidential correspondence
- An agent can produce 20 variants of a practice question in minutes — useful for randomised coursework

In your course design

- Which assessments can you redesign around the assumption students have AI access?
- Which genuinely need invigilation, oral components, or structured defence?
- Assignment briefs should name what’s allowed — silence defaults to “everyone’s guessing”
- Stage permission: foundational exercises without AI, later work with AI + a written or oral defence

Provisional Advice For Students

UG and PGT - metacognition problem most acute, risk losing ‘productive struggle’

- Engage department/school on policy early. “Evolving” is the honest status of most UK-HE AI guidance in 2026
- Declaration policy in your course handbook, not left to the last-minute reminder
- Teach your students to supervise the agent, not just to prompt it. That is the key skill

PhD students - build intuition in years 1-2, leverage in years 3-4

- Do the core foundational exercises in training and first research projects manually
- Friction here builds the mental model of failure modes you will need later to supervise an agent
- Start hoarding working examples as you go (Pattern 1) – future-you will thank present-you!
- Fine to use AI for boilerplate (plotting, file I/O, CI, docstrings) from day one (in my opinion, check supervisor’s view)
- Spec-first becomes essential – novel work means you are the source of novelty, not the agent
- Delegate the mechanical: input scaffolding, API wrangling, figure plotting, benchmarking harnesses
- Keep one regular “no-AI” problem for skill maintenance – recall the Dec-2025 Anthropic comprehension result
- Red/green TDD against relevant domain, not just code: the agent cannot guess your tolerances

Guidance on Responsible Use of AI in Research

- Declare substantive AI use following funder, publisher, and institutional policy: UKRI permits AI usage in proposal writing with caution, Wellcome requires declaration, most publishers want a line in Methods or Code Availability.
- Most existing policies were written for prose, not code. If generated code shaped a result, disclose it.
- You are responsible for every equation, figure, and line of code with your name on it. If an agent contributed, you need to understand and verify that contribution.
- Reproducibility is even more important in the AI age: pinned dependencies, fixed random seeds, good tests. `git log` with Claude trailers is a partial audit trail, I suggest keeping it.
- Never paste confidential material into standard LLMs without agreement of co-authors/collaborators, because of the risk of data leakage: unpublished manuscripts, industry-partner data, student work, proposals or papers you are reviewing. UKRI specifically prohibits AI in proposal review; most publishers prohibit it for manuscript review.
- Agree conventions with your collaborators and co-authors early; set expectations with your students in the course handbook, not after the fact.
- Most UK universities have an institutional AI policy, or are drafting one in 2026 — check yours for approved tools, data-handling rules, and any enterprise-licensed assistant your institution provides.

Summary

Three acceleration mechanisms

- **Rapid prototyping** – explore and fail fast
- **Bridging expertise gaps** – fluency across domains
- **Maintaining codebases** – control technical debt

Four patterns

- Hoard
- Spec first
- Red/green TDD
- Manage context

Four anti-patterns

- Dumping unreviewed code on collaborators
- Trusting output you haven't physically verified
- Stateful notebooks
- Over-engineering by default

What Agentic Coding won't accelerate

- Formulating questions
- Interpreting physical meaning
- Knowing when something is subtly wrong

Writing code is cheap now. Delivering **correct** code – and **knowing** it's correct – is not.
Domain expertise is what turns a capable agent into a useful scientific instrument.

Discussion

In your discipline, what can a student now do with AI that we used to call evidence of understanding?

And what still counts?

Resources

Guides & writing

- **Simon Willison** – Agentic Engineering Patterns (Feb 2026+)
  simonwillison.net/guides/agentic-engineering-patterns/
- **Patrick Mineault** – Claude Code for Scientists (Jan 2026)
  neuroai.science/p/claude-code-for-scientists
- **MIT Missing Semester** – Agentic Coding (2026)
  missing.csail.mit.edu/2026/agentic-coding/
- **mograder** – the case study
  jameskermode.github.io/mograder/

Claude Code plugins

- **obra/superpowers** – skills framework & methodology
  github.com/obra/superpowers
- **marimo-team/marimo-pair** – agents inside a marimo notebook
  github.com/marimo-team/marimo-pair

Slides + source for both decks

github.com/jameskermode/agentic-coding-seminar

Released under CC-BY-4.0 · slides themselves produced with Claude Code (see README).